Predictive Evaluation Competition: Methods Tried, Hosted Results, and Final Selection

Competition Technical Report

June 2026

Abstract

This report summarizes the submission cycle for the Codabench Predictive Evaluation Competition [3]. The competition ranks submissions by hosted negative log-loss, with AUC-ROC as a secondary diagnostic metric. Across 94 parsed hosted rows (92 finished scored rows), the best rounded hosted score improved to -0.57. The selected final submission is `submission_anchor_fam_b60_debias.zip`, with hosted score -0.57, approximate log-loss 0.57, and competition-reported AUC-ROC 0.73. The main empirical conclusion is that compact domain-prior models were the safest no-label baseline, but the successful break below 0.60 log-loss came from using the platform's random revealed labels as an unbiased hidden-category base-rate anchor.

1 Executive Summary

This is a competition submission report, not a course-project writeup. It is written around hosted Codabench evidence and the final ZIP intended for publication in the Predictive Evaluation Competition.

The selected submission is `submission_anchor_fam_b60_debias.zip`. Its hosted negative log-loss was -0.57, which corresponds to ordinary log-loss about 0.57. The competition interface also reported AUC-ROC 0.73; the local `eval_finals.txt` file records the hosted score and submission metadata.

The search first plateaued around -0.62. Compact domain-prior submissions transferred better than AUC stacks, tree blends, forecast ensembles, item-memory models, and hierarchical OOD extensions because they made fewer public-benchmark-specific claims. The extra day changed the diagnosis: the best remaining signal was not another public-data feature, but the random revealed labels supplied by the hosted runtime. The final model estimates the hidden category base rate from those labels and adds a moderate family-aware subject correction.

The original target was hosted log-loss below 0.60, equivalent to displayed negative log-loss above -0.60. The final selected anchor reached that target. The result does not mean local high-AUC evidence was trustworthy; it means the submission finally used hidden-run information in a calibrated way.

2 Problem Setup And Metric

The task is amortized prediction of subject-item correctness. At runtime, the submitted `model.py` receives visible benchmark, condition, subject content, item content, and a small set of previously revealed labels. It must return $p_i = \Pr(y_i = 1)$ for each hidden row [2, 3].

The primary metric is negative log-loss:

$$\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right].$$

This is a negative mean log-likelihood, bounded above by zero, so higher leaderboard values are better. A score of -0.57 is approximately log-loss 0.57. AUC-ROC is secondary: AUC measures ranking, while log-loss also penalizes the absolute probability scale.

The hosted hidden-evaluation setup is central to the result. Public training data came from Measurement DB [1], but hidden test items were served only through the runtime. The starter-kit notes state that hidden test items come from benchmarks not present in public training [2]. Exact item memory and public benchmark-specific priors were therefore structurally risky.

3 Data And Hidden-Evaluation Constraints

The hidden evaluation sampled 5000 items per submission, stratified by organizer data category. The category field was not passed to the predictor. If a submission did not include `labeling.py`, the platform revealed a random sample under the same category budget. If a policy was present, the platform used acquisition scores subject to the hidden category budget.

This made two constraints binding. First, public row holdout was too optimistic because benchmark, subject, condition, and sometimes item overlap remained. Second, active labeling could easily bias the revealed sample. Random labels are noisy, but for base-rate estimation they are valuable because they are approximately unbiased for the hidden category.

4 Methods Tried

4.1 Notation And Shared Calibration

Let s denote a subject, i an item, and $y_{s,i} \in \{0,1\}$ the correctness label. Every submitted method produced a probability $p_{s,i}$, clipped to $[\epsilon, 1-\epsilon]$ to avoid infinite log-loss. Many estimates used shrinkage:

$$\text{shrink}(\bar{y}_g, n_g; c, \tau) = \frac{n_g \bar{y}_g + \tau c}{n_g + \tau},$$

where g is a group, c is a base rate, n_g is support, and τ controls pullback toward c. Several models also used temperature-style calibration:

$$p' = \sigma\left(\alpha + \beta\ \text{logit}(p)\right),$$

where $\beta < 1$ cools the probability spread and $\beta > 1$ sharpens it.

4.2 Global Centers And Subject Priors

The simplest candidates predicted a constant:

$$p_{s,i} = c.$$

Subject-prior variants added a shrunk visible-subject ability estimate:

$$\hat{p}_s = \frac{n_s \bar{y}_s + \tau_s c}{n_s + \tau_s}, \qquad p_{s,i} = (1-\lambda_s)c + \lambda_s \hat{p}_s, \qquad \lambda_s = \frac{n_s}{n_s + \tau_s}.$$

Alias matching connected visible subject descriptions to public subject records. These variants were useful calibration baselines, generally scoring around -0.63 to -0.66, but subject priors alone did not identify hidden benchmark difficulty.

4.3 Benchmark, Condition, And Engineered Priors

Early engineered variants added public benchmark and condition means:

$$p_{s,i} = c + a_s + b_{\text{bench}(i)} + q_{\text{cond}(i)}.$$

The terms were shrunk toward zero. This improved public-slice validation but had poor hidden-transfer logic: benchmarks not present in public training cannot be recognized by exact public benchmark means. These features therefore became fallback calibration signals rather than final decision rules.

4.4 Semantic And Text Difficulty Models

Semantic/text models attempted to infer item difficulty from visible item content and subject descriptors:

$$z_{s,i} = \beta_0 + \beta_s^\top \phi_s(x_s) + \beta_i^\top \phi_i(x_i) + \beta_{si}^\top \phi_{si}(x_s, x_i), \qquad p_{s,i}^{\text{text}} = \sigma(z_{s,i}).$$

Feature maps ϕ represented domain keywords, task-type cues, difficulty markers, subject-family indicators, and semantic similarity features. The text score was usually blended with a conservative prior:

$$p_{s,i} = (1-\eta)p_{s,i}^{\text{prior}} + \eta p_{s,i}^{\text{text}}.$$

The failure mode was calibration. Text features sometimes ranked public rows well, but the absolute probability assigned to unseen hidden benchmark families was unstable.

4.5 Adaptive Labeling Attempts

The competition permits a small number of labels to be revealed before full prediction. Acquisition policies assigned a score $r_{s,i}$, such as uncertainty sampling

$$r_{s,i} = -|p_{s,i} - 0.5|,$$

or coverage sampling over predicted domains. After labels were revealed, simple adjustment models used

$$\Delta_L = \rho(\bar{y}_L - \bar{p}_L), \qquad p_{s,i}^L = \text{clip}(p_{s,i} + \Delta_L, \epsilon, 1-\epsilon).$$

Hosted results did not support aggressive acquisition. A key later lesson is that omitting `labeling.py` was beneficial for the final anchor because the platform-random labels preserved an unbiased base-rate estimate.

4.6 Item Memory And Exact Lookup

Item-memory models treated normalized public item text as a key:

$$\hat{p}_i = \frac{n_i \bar{y}_i + \tau_i p_{s,i}^{\text{fallback}}}{n_i + \tau_i}.$$

This looked strong under row holdout because public validation rows shared item text with training rows. Under benchmark-family holdout, item coverage was effectively zero, matching the hidden setup. Hosted item-memory variants stayed near -0.63 to -0.64, confirming that exact public item memory did not transfer.

4.7 AUC Stacks And Pairwise Rankers

AUC-focused submissions combined component predictors with a logit opinion pool:

$$p_{s,i} = \sigma\left(\alpha + \gamma \sum_m w_m \, \mathrm{logit}(p^m_{s,i})\right), \qquad w_m \geq 0.$$

The scale γ controlled sharpness. Higher sharpness often improved local ranking metrics but increased hidden log-loss risk. Pairwise rankers estimated latent utilities

$$\Pr(y_a > y_b) = \sigma(u_a - u_b),$$

then converted ranks to probabilities. These models were aligned with AUC, not with calibrated log-loss. Hosted AUC/rank variants scored roughly -0.64 to -0.71.

4.8 Tree Models And Forecast Ensembles

Tree models used structured subject, domain, text, public-prior, and interaction features. Forecast ensembles then pooled model outputs:

$$z_{s,i} = \alpha + \sum_m w_m \, \mathrm{logit}(p^m_{s,i}), \qquad p_{s,i} = \sigma(z_{s,i}/T).$$

On public-proxy validation, some forecast optimizers produced log-loss near 0.34 and AUC above 0.928. Hosted results did not transfer, landing around -0.63 to -0.67. The likely cause was overfitting public benchmark and item structure, followed by overconfident probabilities on shifted hidden families.

4.9 Compact Domain Priors

The most stable pre-anchor family used broad subject and item-domain signals. Item text was mapped into coarse indicators $D_k(i)$, and each domain effect was shrunk:

$$\delta_k = \frac{n_k}{n_k + \tau_d}(\bar{y}_k - c), \qquad d(i) = \sum_k D_k(i)\delta_k.$$

Subject ability was similarly shrunk:

$$a_s = \frac{n_s}{n_s + \tau_s}(\bar{y}_s - c).$$

The compact prediction was

$$p_{s,i} = \mathrm{clip}\left(c + w_a a_s + w_d d(i), \epsilon, 1 - \epsilon\right).$$

This family avoided exact item lookup, public benchmark lookup, and high-variance interactions. Multiple compact domain-prior variants reached the early -0.62 plateau, and the final anchor retained this compact formula as the no-label fallback.

4.10 Hierarchical OOD Extensions

Hierarchical OOD variants added subject-domain interactions and conservative revealed-label corrections:

$$p_{s,i} = \text{clip}\left(c + a_s + d(i) + h_{s,d(i)} + \ell_L, \epsilon, 1-\epsilon\right),$$

with

$$h_{s,d} = \frac{n_{s,d}}{n_{s,d} + \tau_h}\left(\bar{y}_{s,d} - c - a_s - d\right).$$

Cold benchmark-family validation suggested improvement, but hosted results ranged from -0.63 to -0.68. The additional interactions likely added enough variance to hurt log-loss on the hidden sample.

4.11 Final Domain Micro-Sweeps

After high-complexity methods degraded, calibration-center sweeps tested whether the hidden base rate was simply lower or higher than 0.60:

$$p_{s,i}(c, \lambda) = \text{clip}\left(c + \lambda(w_a a_s + w_d d(i)), \epsilon, 1-\epsilon\right).$$

Centers from 0.550 to 0.630 and several cooling strengths were submitted. None beat the compact -0.62 plateau. That result made a single global-center correction unlikely to be sufficient.

4.12 Revealed-Label Anchoring

The final successful family used random revealed labels as a hidden-run base-rate estimator. For a target row i, the model constructed a hierarchy of grouping keys:

$$\mathcal{G}(i) = [\text{pooled}, \text{family}(i), \text{signature}(i), \text{benchmark}(i), (\text{benchmark}(i), \text{condition}(i))].$$

The family and signature levels are coarse groupings derived from visible metadata and text. The benchmark and condition levels are populated only from labels revealed during the hosted run.

For each group g, revealed labels define a Laplace-smoothed rate in logit space:

$$m_g = \frac{s_g + \alpha}{n_g + 2\alpha}, \qquad \ell_g = \text{logit}(m_g).$$

Child logits are shrunk toward parent logits:

$$\tilde{\ell}_g = w_g \ell_g + (1 - w_g)\tilde{\ell}_{\text{parent}(g)}, \qquad w_g = \frac{n_g}{n_g + \tau},$$

with $\alpha = 1$ and $\tau = 4$ in the selected variant. This makes the prediction close to the hidden category's observed base rate while limiting overconfidence from small samples.

The model then adds a family-aware subject correction. Public data estimate

$$o_{s,f} = \text{clip}\left(\text{logit}(\hat{p}_{s,f}) - \text{logit}(\hat{p}_f), -c_o, c_o\right).$$

The selected debiased variant subtracts the mean subject offset in the revealed sample and adds the target subject's offset:

$$z_{s,i} = \tilde{\ell}_{g^*(i)} - \beta \bar{o}_L + \beta o_{s,f(i)}, \qquad p_{s,i} = \sigma(z_{s,i}),$$

where $\beta = 0.60$. Offline simulation of the exact runtime with random revealed labels selected the family-aware debiased configuration, and hosted evaluation confirmed it: `submission_anchor_fam_b60_debias.zip` scored -0.57. Nearby anchor variants also showed the risk of over-adjustment, ranging from -0.60 to -0.82.

5 Hosted Results

Table 1: Final selected hosted submission.

Field	Value
ZIP	`submission_anchor_fam_b60_debias.zip`
Hosted submitted time	2026-06-06 07:28
Hosted negative log-loss	-0.57
Approximate log-loss	0.57
AUC-ROC	0.73 (competition reported)
Runtime family	Revealed-label anchored base-rate model with compact domain-prior fallback

Table 2: Hosted negative log-loss by method family. Scores are leaderboard values; higher is better.

Family	Rows	Best score	Median score	Best ZIP
Label-anchored	7	-0.57	-0.60	`submission_anchor_fam_b60_debias.zip`
Compact domain prior	21	-0.62	-0.63	`submission_domain_prior_hidden_cent.zip`
Final domain micro-sweep	9	-0.62	-0.63	`submission_domain_live_mid610.zip`
Alias/scaled priors	15	-0.63	-0.63	`submission_alias_light.zip`
External/domain prior	4	-0.63	-0.63	`submission_external_domain_mid600.zip`
Forecast ensemble	5	-0.63	-0.63	`submission_forecast_robust_logloss_.zip`
Hierarchical OOD	5	-0.63	-0.66	`submission_hier_ood_center600.zip`
Item memory	2	-0.63	-0.64	`submission_item_memory_rank.zip`
AUC/rank stack	6	-0.64	-0.65	`submission_auc_stack_item_push_labe.zip`
Baseline/calibration	7	-0.64	-0.66	`submission_calibrated_constant.zip`
Other	3	-0.64	-0.65	`submission_subject_balanced.zip`
Tree blend	3	-0.64	-0.64	`submission_tree_blend_maxauc_labeli.zip`
OOD gated	2	-0.65	-0.66	`submission_ood_gated.zip`
Semantic/text	3	-0.65	-0.65	`submission_semantic_labeling.zip`

Table 3: All finished submissions tied for the best rounded hosted score in `eval_finals.txt`.

ID	Date	Score	Family	ZIP
782820	2026-06-06 07:28	-0.57	Label-anchored	`submission_anchor_fam_b60_debias.zip`

Figure 1 plots finished hosted scores by submission order. Figure 2 summarizes the best score by method family. Figure 3 shows the search narrative: compact domain-prior candidates established

the -0.62 plateau, higher-complexity candidates mostly failed to improve it, and revealed-label anchoring produced the -0.57 best row.

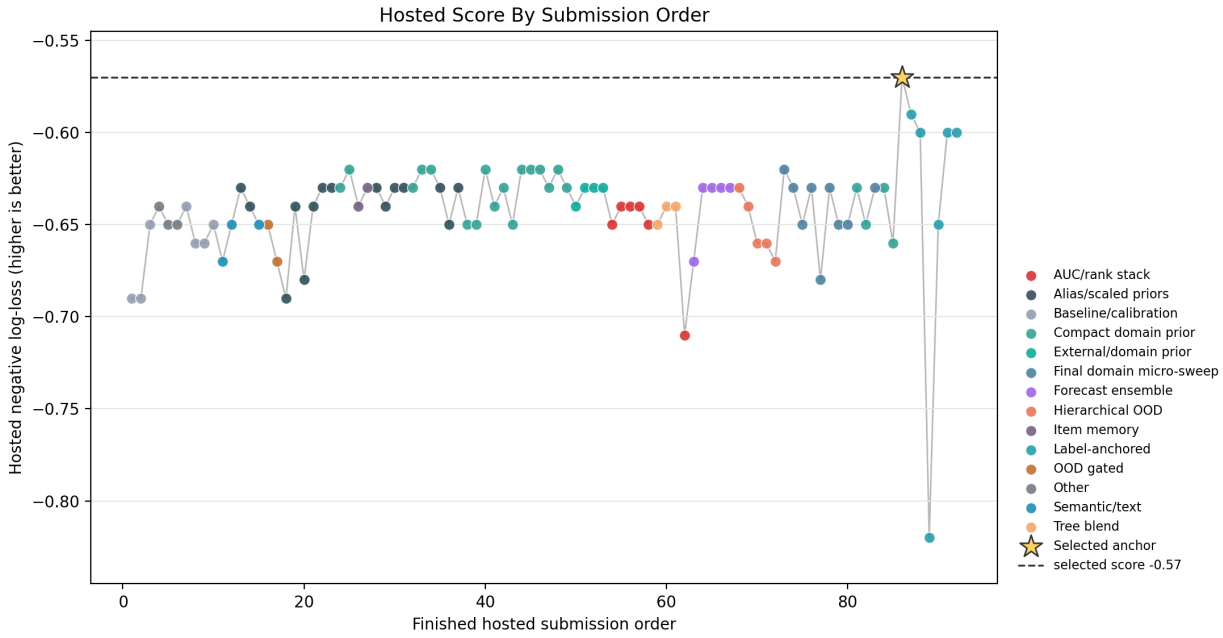

Figure 1: Hosted negative log-loss by finished submission order, colored by method family.

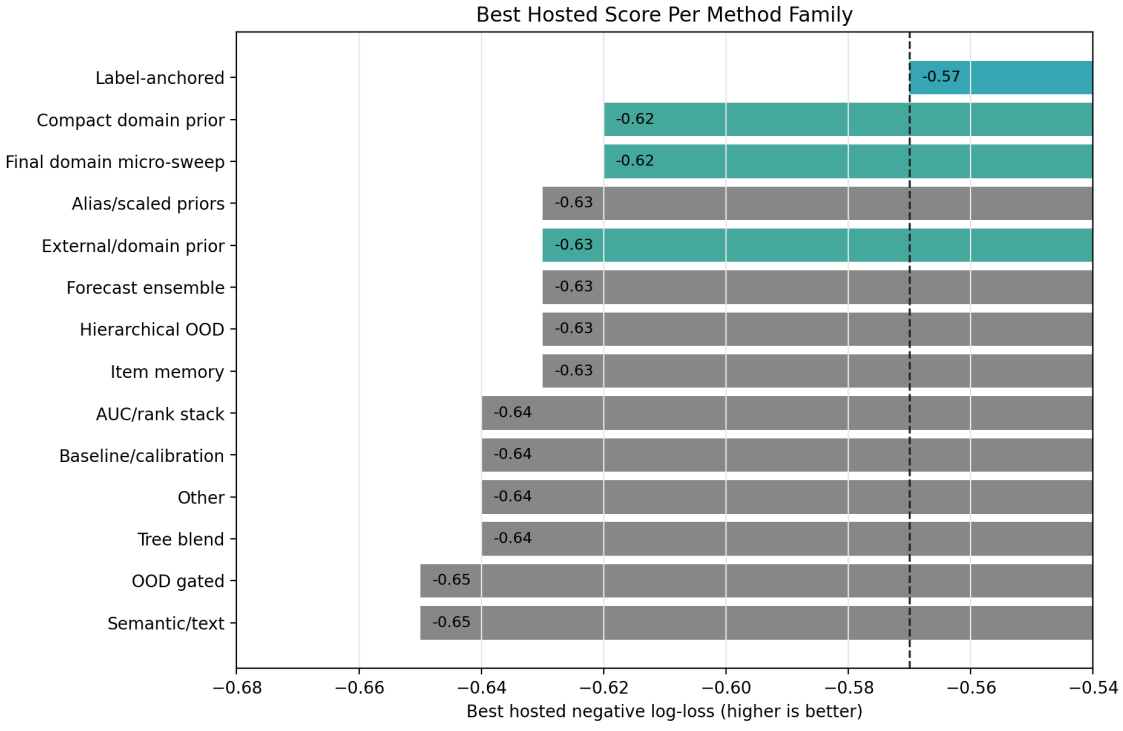

Figure 2: Best rounded hosted score per method family. The dashed line marks the selected score.

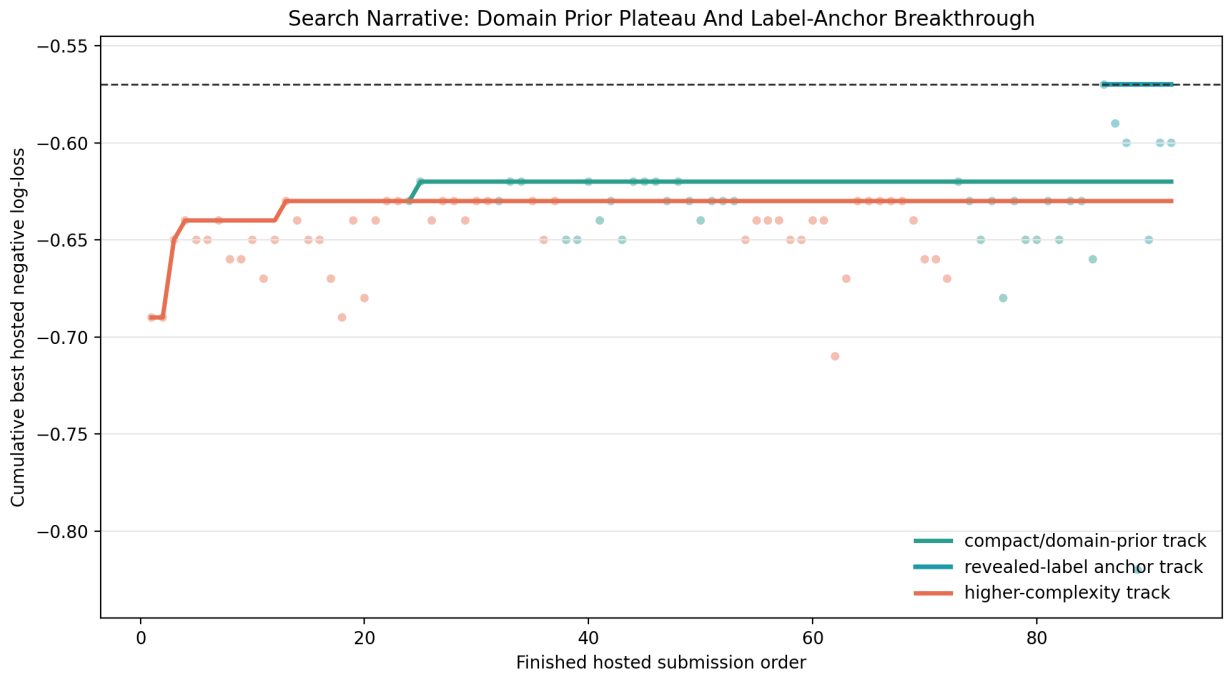

Figure 3: Cumulative best score for compact/domain-prior candidates, higher-complexity candidates, and revealed-label anchor candidates.

6 Why High Local AUC Did Not Transfer

The hosted evidence shows that public-slice AUC was not reliable promotion evidence for this competition.

Validation overlap was too permissive. Public-slice probes sampled from the same public matrix used to build many artifacts, preserving subject, benchmark, condition, and sometimes item overlap. Exact item memory had high row-holdout coverage but zero benchmark-holdout item coverage.

AUC rewarded ranking rather than calibration. AUC-heavy models used wide probability spreads. That can rank familiar rows well while worsening log-loss on shifted hidden families. The strict benchmark-holdout audit already showed weak transfer, with weighted log-loss 0.729388 and weighted AUC 0.537691.

Hidden benchmark families changed the base-rate problem. Models could learn that a public benchmark was easy or hard, but that did not identify the hidden benchmark family's base rate. The compact domain prior was robust because it stayed broad; the anchor improved further because it estimated the hidden-run base rate directly from revealed labels.

7 Final Selected Method

The selected method is `submission_anchor_fam_b60_debias.zip`. Its runtime formula is:

$$p_{s,i} = \sigma\left(\tilde{\ell}_{g^*(i)} - \beta\bar{o}_L + \beta o_{s,f(i)}\right).$$

The term $\tilde{\ell}_{g^*(i)}$ is the shrunk hidden-run base-rate logit from random revealed labels, $o_{s,f(i)}$ is a family-relative subject offset, and $\bar{o}_L$ debiases the revealed sample for subject composition. The ZIP contains only `model.py` and `artifacts/domain_prior.json`. It has no `labeling.py`, no exact item-memory table, no tree model, and no forecast ensemble.

This method achieved the competition target of log-loss below 0.60. It replaced `submission_domain_live_mid600.zip` as the final selection because the updated hosted evidence gave it score -0.57 and competition-reported AUC-ROC 0.73.

8 Lessons Learned And Future Work

Hosted negative log-loss must remain the objective. Public-proxy AUC can diagnose ranking signal, but it should not promote a submission unless cold benchmark-family log-loss and calibration also improve.

The most useful future work is better hidden-run base-rate estimation. That means more robust grouping keys for revealed labels, better small-K variance control, and subject-family offsets trained without benchmark-family leakage. Active labeling should remain disabled unless it beats platform-random labels while preserving an unbiased category base-rate estimate.

9 Clean Publication Repository

The clean publication artifact is organized as `aims-competition` and published at https://github.com/BlakeMasters/aims-competition. It contains the final uploadable ZIP, extracted final submission source, archived alternate submission ZIPs under `unused_submissions`, this report as PDF and LaTeX source, generated figures and tables, hosted-results files, minimal validation tools, and tiny sample files for smoke testing. It intentionally excludes raw benchmark downloads, validation extraction folders, virtual environments, Python caches, agent instructions, documentation folders, and iterative working artifacts.

10 Reproducibility

The hosted-results tables and figures were regenerated from `eval_finals.txt`. The parser found 94 hosted rows and 92 finished scored rows. The best rounded hosted score was -0.57, tied by 1 finished rows in the cleaned hosted result table. The derived CSV files under `report/tables/` provide the exact counts used in Table 2.

References

[1] Aims foundations measurement db dataset. https://huggingface.co/datasets/aims-foundations/measurement-db, 2026. Public training dataset referenced by the starter kit notes.

[2] Predictive ai evaluation challenge starter kit notes. Local competition starter kit file: `asn.txt`, 2026. Defines the runtime interface, hidden-evaluation setup, and primary negative log-loss metric.

[3] Predictive evaluation competition. https://www.codabench.org/competitions/15934/, 2026. Codabench competition page for the hosted Predictive Evaluation Competition.